

Developer & Architecture Guide

Backend pipeline, modules, and data model

Document	4 of 9
Audience	Developers
Revision	Rev 1.0 · 2026
Status	Handoff

Castillo Planset QC — Developer & Architecture Guide

1. Purpose and Scope

This guide is the definitive engineering reference for the **Castillo Planset QC** backend and its analysis pipeline. It is written for the Developer who will maintain, extend, or debug the system. It documents the responsibilities of every module under **backend/app**, the end-to-end analysis flow from PDF upload to persisted findings, the AI provider abstraction, the bounded job queue, the SQLite schema, and how to run and test the system locally.

The application is an AI-assisted QA/QC tool for solar PV plansets. A FastAPI backend (**backend/app**) drives a multi-engine analysis pipeline; a React/Vite frontend (**frontend**) presents the results. It is deployed as a single container on Azure Container Apps, serving both the API and the built SPA from one process.

Status vocabulary. Every finding carries one of four statuses that recur throughout the code and the UI: **Pass** (green), **Fail** (red), **Needs Review** (amber), **Deferred** (purple). A fifth status string, **Overridden / Accepted by QC Engineer**, is produced when the QC Engineer manually overrides an auto-generated finding.

2. Repository Layout — backend/app Modules

Each module below is described by its responsibility. File paths are absolute-from-repo-root for clarity.

backend/app/main.py — HTTP API

The FastAPI application. Defines the ASGI **app**, CORS middleware (currently `allow_origins=["*"]`), an access-logging HTTP middleware that skips the noisy `/api/progress/` and `/artifacts/` polling paths, and a rotating file-log handler writing to `<DATA_DIR>/logs/planset_qc.log` (5 MB × 5 files). It mounts the data directory at `/artifacts` for serving rendered snippets and page previews, and — when `FRONTEND_DIST` points at a built frontend — mounts the SPA at `/` (last, so `/api` and `/artifacts` win). It exposes the analysis, projects, versions, issues, feedback, export, and identity endpoints, and initializes the database on startup. Analysis requests are validated here, then handed to the bounded queue in `jobs.py`; results are polled back via `GET /api/result/{upload_id}`.

Principal endpoints:

Method & path	Purpose
GET <code>/api/health</code>	Liveness probe.
GET <code>/api/me</code>	Signed-in engineer identity (EasyAuth, or dev fallback).

POST /api/analyze	Enqueue a new planset analysis; returns <code>upload_id</code> + <code>status: queued</code> .
POST /api/runs/{run_id}/reanalyze	Enqueue a non-destructive rerun as the next version in the lineage.
GET /api/result/{upload_id}	Fetch a finished run (202 while still processing).
GET /api/progress/{upload_id}	Per-job progress {step, detail, pct}.
GET /api/jobs	Shared activity feed + queue stats.
GET /api/runs, GET /api/runs/{id}	List / fetch runs.
GET /api/runs/{id}/versions	Rerun lineage, oldest → newest.
PATCH /api/runs/{id}	Rename a run.
GET /api/projects, GET /api/projects/{id}	Project browser data.
PATCH /api/issues/{id}	Update a finding's status / override comment.
POST /api/issues/manual	Add a manual finding.
POST /api/issues/{id}/feedback, POST /api/runs/{id}/feedback	Per-finding and per-run feedback capture.
POST /api/parse-project-details, POST /api/parse-supporting-docs	AI-assisted intake of project docs and engineering source documents.
GET /api/export/{run_id}, GET /api/due-diligence-template	XLSX exports.

backend/app/analyzer.py — Orchestration + Sheet/Page Routing

The heart of the pipeline. `analyze_pdf(...)` opens the PDF with PyMuPDF (`fitz`), extracts per-page text and infers each page's **sheet number** and **sheet title** from the title block, parses the cover's **drawing index** (regex first, AI-vision fallback), and runs the procedural drawing-index and title-block checks. It then dispatches the registry-driven `sheet_existence` and `keyword` rules, auto-extracts electrical specs from the planset (cover / SLD / datasheet passes) into a `project_details`-shaped dict plus a per-field **provenance** map, runs the deterministic `electrical_calc` rules, invokes the Gemini/OpenAI vision checks, then the diagram-consistency and cross-sheet-consistency passes, same-referent verification, and finally finding de-duplication. It builds the run summary and returns (`run`, `issues`). This module also owns the sheet-number regex/validation heuristics (excluding wire types, fuse specs, and standards that look like sheet numbers), design-stage auto-detection from title-block revision markers, and all issue-artifact rendering (`render_issue_artifacts` — text-search hit clustering, table-cell/line expansion, highlighted page previews and cropped snippets). `make_issue(...)` is the canonical finding-dict factory used across every engine.

backend/app/gemini_client.py — Provider Abstraction

A thin, provider-agnostic client over **OpenAI** and **Gemini**. `AI_PROVIDER` (`openai` default, or `gemini`) selects the backend; all four public functions — `analyze_page_image`, `analyze_multiple_images`, `analyze_text`, `analyze_document` — share one signature regardless of provider, each taking a `deep: bool` flag. Model selection, retry-on-transient-error (429/503/504), per-run token-usage accounting (`reset_usage/get_usage`), and the optional `OPENAI_SEED` best-effort determinism control all live here. See §4.

backend/app/gemini_analyzer.py — Vision Prompts & Deep-Check Orchestration

Contains every AI vision check (the `check_*` functions) and their prompts. `run_gemini_checks(...)` is the entry point called from `analyze_pdf`; it fans checks out across an internal `ThreadPoolExecutor` (size `AI_PARALLELISM`, default 6) since each call is IO-bound on the model API, wraps each in `_safe_gemini_call` so one failure never blocks the rest, and drains results at the end. It routes heavy reasoning checks (SLD, DC, three-line, relay, cross-sheet, deep electrical, system info) onto the deep model when `use_deep` is true. A `_GLOBAL_INSTRUCTIONS` block appended to every prompt enforces the finding contract: a searchable verbatim `location_text` per finding, distinct locations per page, readable prose evidence, and "Fail only when confident." Pages are rasterized once and cached per (`page`, `zoom`).

backend/app/electrical_calcs.py — Deterministic NEC Calculations

Pure, testable engineering math — no AI. Each `validate_*` function takes the merged `project_details` dict and returns a `CalcResult` dataclass (`status`, `evidence`, `severity` default `medium`, `confidence` default `0.85`). Coverage includes string Voc/Vmp cold and hot limits, fuse sizing, transformer sizing, DC/AC/MV ampacity, EGC and GEC sizing, voltage drop, conduit fill, NEC clearances, module count, total DC/AC, and DC:AC ratio. The registry names a function per rule via `calc_function`; `run_calc(name, pd)` dispatches through the `CALC_FUNCTIONS` table. A calc that returns `Needs Review` because its inputs weren't extracted is promoted to **Deferred** by the analyzer (evidence begins with "Missing "), so missing-input calcs read as "not run" rather than polluting the review list.

Rules Loading — backend/app/rule_registry.py (+ `rules.yaml`, `stage_overrides.yaml`)

Loads the QC rule set from `rules.yaml` into frozen `Rule` dataclasses, cached at import. `RULES_FILE` can point at an alternate YAML (e.g. `rules_v4_draft.yaml`) for A/B testing without code changes. Rules are typed by `check_type` (`keyword`, `sheet_existence`, `electrical_calc`, `manual`, plus V4 vision rules) and are looked up via `get_rules_by_check_type(...)`, `get_category_order()`, etc. `stage_overrides.yaml` supplies each rule a `min_stage` (per-rule override or category default) plus a `disabled_rules` set; `STAGE_ORDER = ("30", "60", "90", "IFC", "AsBuilt")` defines gating order (a rule fires at its stage and all later stages).

backend/app/dedup_findings.py — Multi-Engine De-duplication

`consolidate_overlapping_findings(issues)` collapses findings that flag the **same concept** from **different check engines**. Dense electrical sheets get the same NEC item raised independently by a per-sheet vision check (`ai_gnd_*`), a deep multi-page check (`ai_gnd_deep_*`), and a deterministic keyword check (`gnd_*`). A finding's concept signature is derived from an explicit **NEC 250.x** reference or a specific concept keyword matched against the title (main bonding jumper, GEC, EGC, transformer grounding, DC/AC totals, voltage drop, etc.). A group is merged only when its members span **≥2 distinct engines**; the survivor keeps the **most severe** status and richest evidence, with a consolidation note. Findings with no recognized signature are never touched. Order-preserving.

backend/app/consistency.py & backend/app/diagram_consistency.py — Cross-Sheet Consistency + Conductor Normalization

`consistency.py` compares each spec field's sightings across the planset. `compare_provenance(provenance, page_map)` groups a field's values from every source (cover, SLD, schedule, datasheet pages, user input, supporting docs), normalizes them so unit/format differences don't produce false conflicts (`normalize_value` and the `_normalize_power/_voltage/_awg/_winding/_ratio/_dimension` helpers handle kW/MW, V/kV, AWG/kcmil, winding configs, ratios, and feet-inches dimensions), and emits a **multi-location** finding only for genuine planset-vs-planset disagreements. Ambiguous textual conflicts are routed to `reconcile_text`, which uses a vision call to decide. `diagram_consistency.py` reads labeled values **off the drawings** — equipment tags-with-ratings, dimension callouts, schedule cells, ratings inside one-line symbols — that never reach the text layer. `run_diagram_consistency(pages, provenance, render=, ai_call=)` selects diagram sheets (one-lines, layouts, schedules, details), extracts annotations, folds recognized labels into `provenance` (so the deterministic comparator catches them — the de-dup mechanism), and returns open-ended cross-checks as findings. It deliberately reuses `consistency.normalize_value`, `_group_sightings`, `_build_finding`, and `reconcile_text` so both engines produce findings in the same multi-location shape.

backend/app/supporting_docs.py — Engineering Source Documents

Ingests supporting docs (CESIR, PVSyst, ampacity studies, interconnection agreements, datasheets). `process_document(filename, data)` reads PDF/XLSX/DOCX/image/text, `classify(...)` and `_subclassify_datasheet(...)` type it, and a type-specific extractor pulls specs into a `SupportingDoc`. `project_details_from_supporting_docs(...)` produces gap-filling `project_details` (used by `POST /api/analyze` to auto-merge datasheet specs — user-entered values always win). `build_evidence_context(...)` renders an evidence block appended to every vision prompt so the model treats supporting-doc values as the authoritative reference.

backend/app/jobs.py — Bounded Analysis Queue

A single-replica, in-memory job registry backed by a `ThreadPoolExecutor` of `ANALYZE_CONCURRENCY` workers (default 2). Every new run or rerun is `submit(...)`-ed here instead of spawning an unbounded thread. Tracks each job's lifecycle and originator so all signed-in engineers see one shared activity feed. See §5.

backend/app/db.py — Schema & Persistence

Owens the SQLite schema, connections, migrations, and all read/write helpers. `DATA_DIR` is `PLANSET_DATA_DIR` (Azure: `/home/data`) or `backend/data` locally; the DB file is `planset_qc.sqlite3`. Because the DB may live on an Azure Files/SMB share that lacks POSIX byte-range locks, the connection URI uses `nolock=1` and `every` access is serialized through a process-wide reentrant `_DB_LOCK` — correct only for the single-replica deployment. See §6.

backend/app/exporter.py — XLSX Export

Builds the two Castillo-branded workbooks with `openpyxl`: the per-run QC report (`export_run_to_excel`) with a Summary sheet (metrics + per-category status breakdown) and an Issues sheet (status-colored rows using the same green/red/amber/purple/blue convention, embedded snippet images, and a **Locations** column that renders cross-sheet-conflict findings as `E-100 p7: 2250 kVA; cover p1: 2000 kVA`), and the blank due-diligence intake template (`export_due_diligence`).

backend/app/auth.py — Identity

Resolves the signed-in engineer from Azure Container Apps EasyAuth (Microsoft Entra) headers (`X-MS-CLIENT-PRINCIPAL*`), which the sidecar injects and client copies are stripped, so they are trustworthy. `current_user` is the FastAPI dependency returning `{email, user_id, name}`. Locally (no sidecar) it falls back to the `DEV_USER_EMAIL` env var.

Supporting modules

`progress.py` — in-memory `{step, detail, pct}` progress store keyed by `upload_id`.
`rule_registry.py`'s siblings `v4_engine.py` (V4 vision-rule engine and the Other-Electrical catch-all titles), `checklist.py`, `training_docs.py`, and `xlsx_template_sniffer.py` support rule ingestion and template handling.

3. End-to-End Analysis Flow

`POST /api/analyze` accepts the PDF plus optional `project_name`, `project_id`, `run_name`, `project_details` (JSON), `supporting_docs` (JSON), `use_deep`, `design_stage`, and `engineer_name`. The request handler validates inputs, auto-merges datasheet-extracted specs into `project_details` (user values win), resolves identity and the project link, saves the PDF under

<DATA_DIR>/runs/<upload_id>/, and **enqueues** the work on `jobs.submit(...)`, returning `{upload_id, status: "queued"}` immediately. A worker thread later runs `_run_analysis_bg`, which calls `analyze_pdf`. Inside `analyze_pdf` the ordered steps are:

1. **Open & parse** — `fitz.open`, then `extract_pages` builds a `PageInfo` per page with text, inferred sheet number, and inferred sheet title.
2. **Design stage** — if the caller didn't set one, `detect_design_stage` scans the first pages' title blocks for revision markers (`AsBuilt` > `IFC` > `IFP/60` > highest %).
3. **Drawing index** — `parse_drawing_index` (regex) on the combined cover text; if coverage < 50%, `parse_drawing_index_ai` (vision) supersedes it.
4. **Procedural checks** — drawing-index existence / sheet-match / no-extras / order, then title-block sheet-number and sheet-title checks, each emitting `make_issue(...)` findings with rendered artifacts.
5. **Registry `sheet\_existence` rules** — match required sheets by title keywords (present ⇒ Pass; indexed-but-missing ⇒ Fail).
6. **Registry `keyword` rules** — search the rule's scope (cover / all / title-matched pages) for required keywords; Pass when `min_matches` met, else Needs Review.
7. **Spec auto-extraction** — `extract_specs_from_pages` runs three vision passes (cover system-info at 2x zoom, SLD + cable/conduit schedule, then per-datasheet pages), producing the merged `project_details` and a per-field `provenance` list `{value, source}`. User-entered `project_details`, plus supporting-doc specs, are also recorded into provenance.
8. **Electrical calcs** — for each `electrical_calc` rule, `run_calc(rule.calc_function, merged_pd)`; missing-input Needs Review results are flipped to **Deferred**.
9. **Vision deep checks** — `run_gemini_checks(...)` fans out the `check_*` functions in parallel and returns their findings; per-call timings and token usage are captured.
10. **Diagram-consistency pass** — `run_diagram_consistency` reads labeled drawing values, folds recognized ones into `provenance`, returns open-ended cross-checks.
11. **Cross-sheet consistency** — `compare_provenance` flags normalized planset-vs-planset conflicts as multi-location findings; diagram findings already covered by a provenance-based finding are dropped.
12. **Same-referent verification** — candidate conflicts are confirmed with a bounded number of vision calls (cap 15) that the disagreeing values describe the same engineering quantity; beyond the cap, conflicts emit unverified as Needs Review (never auto-dropped).
13. **De-duplication** — `consolidate_overlapping_findings` collapses multi-engine same-concept duplicates, keeping the most severe status.
14. **Summarize** — status counts, per-category breakdown, `page_sheet_map`, timings, gemini usage, resolved `design_stage`, and duration are assembled into `summary`; the run dict is returned.

Back in `_run_analysis_bg`, orchestration fields (attribution, `project_id`, and rerun-versioning keys) are stamped onto the run; a fresh analysis is `insert_run`-ed, while a rerun (predecessor present) is versioned atomically via `save_run_version`. Progress hits `done/100`, and the saved run is cached for `GET /api/result/{upload_id}` to return.

4. AI Provider Abstraction

`gemini_client.py` isolates every model call behind four functions. Provider and models are chosen from environment variables read at import (the seed is read at call time):

Variable	Default	Meaning
<code>AI_PROVIDER</code>	<code>openai</code>	<code>openai</code> or <code>gemini</code> .
<code>OPENAI_MODEL</code>	<code>gpt-4.1-mini</code>	Standard (per-page) model.
<code>OPENAI_MODEL_DEEP</code>	falls back to <code>OPENAI_MODEL</code>	Heavier model for deep reasoning passes.
<code>GEMINI_MODEL</code>	<code>gemini-2.5-flash</code>	Standard Gemini model.
<code>GEMINI_MODEL_DEEP</code>	falls back to <code>GEMINI_MODEL</code>	Deep Gemini model.
<code>OPENAI_SEED</code>	<code>unset</code>	Optional best-effort determinism (see below).
<code>AI_TIMEOUT</code>	<code>90 (s)</code>	Per-request timeout.
<code>AI_RETRIES</code>	<code>2</code>	Retry attempts on transient errors.
<code>AI_PARALLELISM</code>	<code>6</code>	Vision-check thread pool size in <code>gemini_analyzer</code> .

Note: the code default and `.env.example` both ship `gpt-4.1-mini` / `gpt-4.1` as the OpenAI standard/deep models. Because the model IDs are env-driven, production can point `OPENAI_MODEL` / `OPENAI_MODEL_DEEP` at any current OpenAI model without a code change.

Standard vs deep selection. Each public function takes `deep: bool`. `_pick_openai_model(deep)` / `_pick_gemini_model(deep)` return the deep model when `deep` is true, else the standard model. `run_gemini_checks` decides per check whether deep is warranted, and further gates it on the run-level `use_deep` flag (`_deep(want) = want and use_deep`), so `use_deep=false` forces every call onto the cheap standard model.

OpenAI vs Gemini dispatch. Each public function branches on `AI_PROVIDER` and calls the matching `_openai_*` or `_gemini_*` implementation. OpenAI images are sent as base64 `data:` URLs with `detail:`

"high"; Gemini uses `Part.from_bytes.analyze_document` for a PDF under OpenAI degrades to a text prompt (OpenAI chat vision takes images only).

Retries & usage. `_call_with_retry` retries on HTTP 429/503/504 with linear backoff ($3 \times \text{attempt}$ seconds), up to `AI_RETRIES`; other errors propagate. `_track_openai` / `_track_gemini` accumulate prompt/completion/total tokens and API-call counts under a lock; `reset_usage()` is called before each analysis and `get_usage()` is stored in the run summary.

Best-effort determinism. Setting `OPENAI_SEED` (an int) makes `_openai_sampling_kwargs()` pass `seed` to every OpenAI chat completion, so identical inputs return near-identical output — useful for reproducing a run or generating clean before/after regression diffs. This is **best-effort only**: an OpenAI model or infra change (reflected in `system_fingerprint`) can still shift results, and it does not resolve genuine model-judgment differences on ambiguous drawings. `temperature` is intentionally not exposed because the reasoning-class models ignore or reject it. Unset $\Rightarrow$ normal sampling (unchanged default behavior). There is no Gemini equivalent.

5. Bounded Job Queue (`jobs.py`)

Every analysis is submitted to a shared, in-memory registry backed by one `ThreadPoolExecutor`:

- **Concurrency.** `CONCURRENCY = max(1, int(ANALYZE_CONCURRENCY or 2))`. At most N analyses run at once; the rest wait in the executor queue with status `queued`. Kept low by default because a single replica shares one API key and CPU, and too many parallel deep runs thrash memory and risk rate limits.
- **Lifecycle / statuses.** Each `Job` moves `queued` $\rightarrow$ `running` $\rightarrow$ `done` | `error`. `submit(job_id, kind, meta, fn)` registers the job (status `queued`), sets initial progress, and hands `fn` to the executor. `_run_wrapped` flips it to `running`, executes `fn` (the analysis, which returns the saved run dict or `{"error": ...}`), and records the terminal status plus `run_id` or `error`.
- **Registry.** `_jobs` (`id` $\rightarrow$ `Job`) and `_order` (submission order) under a reentrant lock. `Job` carries `id` (`== upload_id`), `kind` (`analyze/reanalyze`), `project_name`, `run_name`, `started_by/created_by`, timestamps, `run_id`, and `error`.
- **Feed & stats.** `list_jobs()` returns every tracked job oldest-first with a 1-based `queue_position` for waiting jobs and live progress merged from `progress.py`; it drives `GET /api/jobs`. `stats()` returns `{concurrency, queued, running}`. Finished jobs are retained for the feed by `_prune_locked()` up to 40 jobs / 30 minutes, then dropped.
- **Single-replica constraint.** The registry and queue are in-process. This matches the current deployment (SQLite on Azure Files, one always-on replica). Scaling to multiple replicas requires

moving this state to a shared backing (DB or a real queue); the `/api/jobs` contract can stay the same.

Note this is a **second, outer** concurrency layer: `ANALYZE_CONCURRENCY` bounds how many whole analyses run at once, while `AI_PARALLELISM` (inside `run_gemini_checks`) bounds how many vision calls run concurrently within one analysis.

6. Database Schema (db.py)

SQLite at `<DATA_DIR>/planset_qc.sqlite3`. Tables are created and migrated idempotently in `init_db()`; additive columns are `ALTER TABLE`-added on startup so older DBs upgrade in place. `_backfill_projects_and_versions` runs every startup but only touches NULL columns.

runs

Core columns: `id` (PK), `project_name`, `original_filename`, `created_at`, `pdf_path`, `page_count`, `summary_json`, `status_counts_json`, `categories_json`, `project_details_json`, `engineer_name`.

Feature columns added later:

Column	Purpose
<code>project_id</code>	Links the run to a <code>projects</code> row (app-enforced FK).
<code>design_stage</code>	Promoted out of <code>summary_json</code> so it is queryable (<code>30/60/90/IFC/AsBuilt</code>).
<code>created_by</code> , <code>created_by_id</code>	Attribution: engineer email and Entra object id.
<code>parent_run_id</code>	Predecessor run in a rerun lineage.
<code>root_run_id</code>	Lineage root (defaults to the run's own id).
<code>version</code>	1-based version within the lineage.
<code>is_latest</code>	Exactly one current run per lineage (1/0).
<code>run_name</code>	Optional human-friendly label (falls back to filename in the UI).

`summary_json` holds the full run summary object (sheet counts, `missing_from_pdf`, `extra_sheets`, `gemini_usage`, `page_sheet_map`, `call_timings`, `deep_mode`, `design_stage`, `supporting_docs`, and — when present — per-field `provenance`).

issues

`id` (PK), `run_id` (FK), `category`, `item_key`, `title`, `description`, `severity`, `status`, `auto_status` (the AI's original verdict, preserved across QC-Engineer overrides), `page_number`, `bbox_json`,

`snippet_path`, `page_preview_path`, `evidence`, `confidence`, `override_comment`, `source_doc_filename` / `source_doc_page` / `source_doc_excerpt` (supporting-doc provenance), `locations_json` (per-location list for multi-location cross-sheet-conflict findings; NULL for normal findings), `created_at`, `updated_at`.

projects

`id` (PK), `name`, `number`, `created_at`, `created_by`. Created lazily at upload (`get_or_create_project`, case/whitespace-insensitive name match) and synthesized during backfill from distinct run names.

issue_feedback & run_feedback

`issue_feedback` captures per-finding signal distinct from override status (`tags_json`, `comment`, `engineer_name`). `run_feedback` captures a per-run rating (`saved_time` / `even` / `cost_time`) plus optional comment; insert-only so rating history is preserved.

Versioning helpers

`save_run_version(run, issues, predecessor_run_id)` derives `version/root_run_id/parent_run_id` from the lineage's **current tip** (not the predecessor passed in), inserts the new run as latest, and demotes every other run in the lineage — the whole read-compute-insert-demote sequence holds `_DB_LOCK`, so concurrent reruns are serialized and can't both claim the same version or both stay latest. `get_run_versions`, `list_projects`, and `get_project_detail` provide the project/version read models.

7. Running Locally

Backend

- Create `backend/.env` from `backend/.env.example` and set at least `AI_PROVIDER` and the matching API key:


```
AI_PROVIDER=openai OPENAI_API_KEY=sk-... # never commit; .env
is gitignored OPENAI_MODEL=gpt-4.1-mini # OPENAI_MODEL_DEEP=gpt-4.1 #
optional heavier deep model # OPENAI_SEED=42 # optional best-
effort determinism # DEV_USER_EMAIL=you@castillope.com # local identity (no
EasyAuth sidecar) # PLANSET_DATA_DIR=... # defaults to backend/data
```
- Install dependencies and start the API (from `backend/`):


```
bash pip install -r requirements.txt python -m uvicorn app.main:app --reload --port 8000
```

 The SQLite DB, uploaded PDFs, rendered artifacts, and logs are written under `PLANSETDATADIR` (default `backend/data`). Leave `FRONTEND_DIST` unset locally so the API does not try to serve a built SPA.

Frontend

From `frontend/`:

```

npm install
npm run dev      # Vite dev server on :5173, proxying to the API on :8000
npm run build    # tsc -b && vite build → dist/ (what the container serves)
npm run preview  # serve the production build locally

```

Useful runtime tuning knobs

`ANALYZE_CONCURRENCY` (default 2), `AI_PARALLELISM` (default 6), `AI_TIMEOUT` (90s), `AI_RETRIES` (2), and `RULES_FILE` (swap in an alternate rule YAML) can all be set in the environment.

8. Test Scripts (backend/scripts)

The `backend/scripts` directory holds targeted test and regression harnesses, run directly with Python from `backend/` (e.g. `python scripts/test_jobs.py`). Notable scripts:

Script	Covers
<code>test_jobs.py</code>	Bounded queue: lifecycle, queue positions, retention.
<code>test_consistency.py</code>	Cross-sheet value-consistency normalization and conflict emission.
<code>test_diagram_consistency.py</code>	Drawing-annotation extraction and diagram cross-checks.
<code>test_dedup_findings.py</code>	Multi-engine same-concept consolidation.
<code>test_referent_verification.py</code>	Same-referent conflict verification path.
<code>test_projects_versioning.py</code>	Projects, rerun lineage, <code>is_latest</code> invariants.
<code>test_stage_gating.py</code>	<code>min_stage</code> rule gating.
<code>test_page_targeting.py</code>	Issue-artifact text-search hit targeting.
<code>test_prompt_fixes.py</code>	Prompt/rule false-positive fixes.
<code>test_seed_option.py</code>	<code>OPENAI_SEED</code> sampling-kwarg behavior.
<code>regression_all_plansets.py</code> , <code>regression_v4.py</code> , <code>regression_snapshot.py</code> (+ <code>regression_baseline.json</code>)	Full-planset regression scoring against a saved baseline.
<code>diagnose_wellington_nr.py</code> , <code>verify_tuning_bishop.py</code> , <code>bucket_gonzo_*.py</code> , <code>compare_gonzo_findings.py</code> , <code>extract_gonzo_annotations.py</code>	Planset-specific diagnostics used during tuning.
<code>ingest_v4_checklist.py</code> , <code>tag_training_docs.py</code> , <code>merge_tag_batches.py</code> , <code>extract_stage_sheets.py</code>	Rule-ingestion and training-data tooling.

Because AI output is only **best-effort deterministic** (via `OPENAI_SEED`, and even then subject to model/infra drift), the regression harnesses compare against a stored baseline snapshot rather than asserting exact model text; treat their diffs as review signals, not hard pass/fail gates, when a model version changes.

9. Deployment Notes for the Developer

CI/CD lives in `.github/workflows`: `deploy.yml` builds and ships on push to `master` (production), `deploy-staging.yml` on push to `staging` (sandbox). Both authenticate to Azure via OIDC, build the image with `az acr build`, and roll it out with `az containerapp update`, parameterized by the `AZURE_RESOURCE_GROUP`, `AZURE_CONTAINERAPP_NAME` / sandbox app name, and `ACR_NAME` secrets. The container serves the API and the built SPA from one process (`FRONTEND_DIST` set in the image); the image is pulled by the Container App's managed identity from ACR. All secrets (API keys, identity config) are Container App application settings — never committed. Because both the SQLite DB (on the Azure Files mount at `/home/data`) and the in-memory job queue assume a single replica, do not scale the Container App beyond one replica without first migrating persistence and the queue to shared backings.

Primary source files backing this guide (all under `backend/app`): `main.py`, `analyzer.py`, `gemini_client.py`, `gemini_analyzer.py`, `electrical_calcs.py`, `rule_registry.py`, `dedup_findings.py`, `consistency.py`, `diagram_consistency.py`, `supporting_docs.py`, `jobs.py`, `db.py`, `exporter.py`, `auth.py`, `progress.py`.